

基于自监督与弱监督学习的 图像异常检测论文复现报告

GANomaly、MF-GANomaly、Mask Encoders 与 TS-GAN

复现论文： 基于自监督与弱监督学习的图像异常检测研究

学生姓名： [姓名]

学号： [学号]

专业与年级： [专业、年级]

指导教师： [导师姓名]

学院： [学院名称]

完成日期： [年月日]

[学校名称]

摘要

本文复现步兆军硕士学位论文《基于自监督与弱监督学习的图像异常检测研究》中的四种方法：GANomaly、MF-GANomaly、Mask Encoders 与 TS-GAN。实验以 MVTec AD 的 Grid 类别为对象，将原始图像固定裁为四个象限后输入网络，使用图片级 AUROC 评价异常排序能力，并同时报告四块最大值与平均值两种聚合方式。四种模型均完成训练与测试：GANomaly 的最佳 AUROC 为 0.7026，MF-GANomaly 为 0.7703，Mask Encoders 为 0.9365，TS-GAN 为 0.9534；其中 TS-GAN 使用 10 张异常图进行弱监督训练，测试时已排除这些样本。进一步按照论文描述实现整体 PSNR 与滑动窗口局部 PSNR 作差的异常热图，在窗口边长 8、步长 1 的复现设定下，TS-GAN 在 Grid 上取得 0.9683 的额外 Pixel AUROC。为检验跨域泛化能力，本项目还将 Grid 检查点直接用于 AITEX-AFID：图片级最大、平均聚合 AUROC 分别降至 0.4463 和 0.5482，Pixel AUROC 为 0.7472、Pixel AP 为 0.0060。随后在按原图隔离的 AITEX 固定划分上重新训练 TS-GAN，图片级最大聚合 AUROC 提升至 0.8173、AP 提升至 0.9368，但整体 Pixel AUROC 仅为 0.5213、Pixel AP 为 0.0049。结果说明目标域训练能够明显改善整图识别，却不会自动带来可靠的精确定位。当前实验仅使用随机种子 42，且预处理、测试集组成与部分网络细节包含复现假设，因此结果不能直接证明本实现优于论文，仍需补充多随机种子实验。

关键词：图像异常检测；GANomaly；自监督学习；弱监督学习；MVTec AD；生成对抗网络

报告使用说明

本文档已根据论文、项目代码、配置文件和现有实验输出写成初稿。灰色方括号主要保留在封面个人信息和附录中的关键代码阅读与理解部分。独立验证同时报告总体指标、分类结果、优秀案例和失败案例，不只展示表现最好的样本。

目录

摘要	I
报告使用说明	I
1 论文与复现任务概述	1
1.1 原论文基本信息	1

1.2	研究问题	1
1.3	复现目标与范围	1
1.4	完成情况与复现边界	2
2	复现环境与数据	2
2.1	软硬件环境	2
2.2	训练与复现数据	3
2.3	未参与训练数据的独立验证集	3
2.4	预处理与数据增强	4
3	关键模型介绍	4
3.1	GANomaly	4
3.1.1	基本原理	4
3.1.2	训练目标与异常评分	5
3.2	MF-GANomaly	6
3.2.1	基本原理	6
3.2.2	关键公式	8
3.3	Mask Encoders	8
3.3.1	基本原理	8
3.3.2	关键公式	11
3.4	TS-GAN	12
3.4.1	基本原理	12
3.4.2	关键公式	14
3.4.3	论文式像素定位方法	14
3.5	四个模型的关系	15
4	统一符号与评价指标	15
4.1	符号定义	15
4.2	评价指标	16
5	实现过程与训练设置	16
5.1	项目结构	17

5.2	统一超参数	17
5.3	训练流程	17
5.4	与论文不一致或未公开的实现细节	18
6	MVTec Grid 主实验结果	18
6.1	总体评价问题与对照关系	19
6.2	Grid 首轮复现结果	19
6.3	TS-GAN 滑动窗口 PSNR 像素定位实验	20
7	扩展实验一：AITEK 零样本外部验证	21
7.1	实验目的与数据协议	21
7.2	图片级结果	21
7.3	像素级总体与分类别结果	22
7.4	表现较好的案例	23
7.5	表现不理想的案例	23
7.5.1	误报案例	23
7.5.2	漏报案例	23
7.6	零样本测试结果小结	23
8	扩展实验二：AITEK 域内重新训练	24
8.1	实验目的与数据划分	24
8.2	图片级结果	24
8.3	像素级结果	25
8.4	域内训练案例分析	26
8.5	域内实验小结	26
9	结果讨论	27
9.1	与论文结果的差异	27
9.2	为什么 AITEK 域内结果仍低于 Grid	27
9.3	模型优势与不足	28
9.4	实验有效性威胁	29

10 结论与后续工作	29
10.1 主要结论	29
10.2 后续工作：可以进一步尝试的事项	30
参考文献	31
A 关键代码整理与理解	32
A.1 代码片段记录表	32
A.2 GANomaly：公共基线网络	32
A.3 MF-GANomaly：只列相对基线的增量	33
A.4 Mask Encoders：遮挡补全与两阶段迁移	34
A.5 TS-GAN：单跳连、双生成器与共享判别器	35
A.6 公共训练核心：先更新判别器，再更新生成器	36
A.7 损失函数、异常评分与整图聚合	37
B 完整配置与复现实验命令	38
B.1 配置文件	38
B.2 运行命令	39
B.3 结果文件清单	39

1 论文与复现任务概述

1.1 原论文基本信息

表 1 原论文信息

项目	内容
论文题目	基于自监督与弱监督学习的图像异常检测研究
作者与年份	步兆军，东南大学工学硕士学位论文，2022 年
研究任务	在正常样本充足、异常样本稀缺且类型不确定的条件下，实现图片级异常识别与像素级异常定位
复现模型	GANomaly、MF-GANomaly、Mask Encoders、TS-GAN
论文主要数据集	MNIST、MVTec AD（Grid、Carpet、Wood 等）与视网膜 OCT 数据
论文主要结果	图片级 AUC、异常热图、PSNR 与 SSIM

1.2 研究问题

工业异常检测的输入是一张待检图像，输出是图片是否异常的分数；在需要定位时，还应输出缺陷可能出现的位置。现实生产中正常产品数量多、容易收集，而异常样本较少且缺陷形式难以穷举，因此不能简单依赖大量有标签异常图进行普通分类。论文围绕三个困难展开：GANomaly 对细小纹理异常的特征提取不足，重建图质量会限制定位效果，以及少量已有异常图没有被充分利用。为此论文依次引入多尺度浅层特征、自监督遮挡补全和弱监督双流结构。

1.3 复现目标与范围

1. 跑通 GANomaly 的生成器、判别器、交替优化和异常评分流程，建立可重复的基线。
2. 实现 MF-GANomaly、Mask Encoders 与 TS-GAN，并比较三种改进策略在 Grid 类别上的表现。
3. 在不夸大现有结果的前提下记录复现假设，并为独立数据验证、像素级定位和多随机种子实验预留规范协议。

1.4 完成情况与复现边界

表 2 复现任务完成情况

序号	任务	状态	证据或说明
1	GANomaly 基线训练与评估	已完成	runs/ganomaly_grid_four_crops_seed42/
2	MF-GANomaly 训练与评估	已完成	runs/mf_ganomaly_grid_paper_l2_seed42/
3	Mask Encoders 两阶段训练与评估	已完成	runs/mask_encoders_grid_seed42/
4	TS-GAN 两阶段训练与评估	已完成	runs/ts_gan_grid_seed42/
5	AITEX-AFID 独立外部验证	已完成	TS-GAN 零样本图片级与像素级评价，结果见 runs/ts_gan_grid_seed42/stage2_abnormal/external_aitex/
6	AITEX-AFID 域内训练与留出测试	已完成	按原图划分且重新训练 TS-GAN，结果见 runs/ts_gan_aitex_seed42/stage2_abnormal/aitex_domain_evaluation/

2 复现环境与数据

2.1 软硬件环境

表 3 实验环境

项目	配置
操作系统	Microsoft Windows 11 家庭中文版，64 位（版本 10.0.26200）
CPU	Intel Core i7-14650HX
GPU 与显存	NVIDIA GeForce RTX 5070 Laptop GPU，8151 MiB
内存	15.7 GiB（系统可见容量）
Python	3.11.15
PyTorch 与 CUDA	PyTorch 2.11.0+cu128，CUDA Runtime 12.8
主要依赖	torchvision 0.26.0+cu128，scikit-learn 1.9.0，scikit-image 0.26.0，NumPy 2.4.6，Pillow 12.3.0，PyYAML 6.0.3
AITEX 训练服务器	Ubuntu 22.04.3 LTS，RTX 4090（24 GiB），Python 3.10.14，PyTorch 2.3.1，CUDA 可用
代码版本	Git 提交 029a91c，分支 docs/final-reproduction-results

2.2 训练与复现数据

表 4 训练和测试数据划分

数据集	类别	训练正常	训练异常	测试	用途
MVTec AD	Grid	264	0 或 10	78 或 68	基线复现； TS-GAN 排除用于弱监督训练的 10 张异常图
AITEX-AFID	织物纹理	35（仅校准）	0	212	TS-GAN 零样本外部验证；不重训练
AITEX-AFID	织物纹理	84	9	126	TS-GAN 域内训练；另用 28 张正常图校准阈值

2.3 未参与训练数据的独立验证集

基本原则：该数据集不能用于训练、选择模型、确定阈值或调超参数。若数据域与 MVTec AD 差异较大，应同时报告“直接迁移”结果和“经过适配后”结果，二者不能混写。

表 5 独立验证集记录

项目	待填写内容
数据集名称与来源	AITEX Fabric Image Database (AFID)，来自 AITEX 官方公开页面
选择理由	与 MVTec Grid 同属规则纹理缺陷，但布料材质、纹理周期、图像尺度和采集域均不同，可检验跨数据集泛化能力
正常与异常数量	正常图 141 张，其中 35 张仅用于阈值校准、106 张用于测试；异常图 106 张。像素评价排除缺少掩码的 0100_025_08.png，使用 105 张异常图
是否与训练集重合	数据来源与 MVTec AD 完全独立，未用于模型训练、模型选择或参数调整
图像尺寸与设备	本实验将原始长条织物图像切为不重叠的 256×256 图块，再缩放为模型输入的 64×64 ；采集设备参数未纳入本次实验记录
标签形式	正常/异常图片级标签；105 张异常图具有像素级二值掩码，两个多掩码样本先取掩码并集
预处理	图块缩放为 64×64 、转三通道并归一化到 $[-1, 1]$ ；像素热图拼回原图后逐图归一化到 $[0, 1]$
阈值来源	每种织物各取 5 张正常图，共 35 张；仅根据其分数的 0.99 分位数确定阈值，未使用异常测试标签调阈值

2.4 预处理与数据增强

项目将每张 Grid 图像固定裁成左上、右上、左下和右下四个不重叠象限。GANomaly、MF-GANomaly 与 TS-GAN 将每个裁剪块缩放为 64×64 ，Mask Encoders 缩放为 128×128 ；随后转为三通道张量，并按通道归一化到 $[-1, 1]$ 。训练与测试使用相同的裁剪、缩放和归一化，以避免输入分布因处理流程不同而发生变化。本轮未加入随机翻转、颜色扰动等额外增强。训练阶段 264 张正常原图因此对应 1056 个裁剪样本；测试时先对四块分别评分，再计算整图最大值和平均值。

3 关键模型介绍

3.1 GANomaly

3.1.1 基本原理

GANomaly 是一种只使用正常图片训练的异常检测模型。它假设模型充分学习正常数据的纹理规律后，能够稳定地完成“编码-重建-再次编码”；当输入偏离正常分布时，重建过程及前后两次编码会出现更明显的不一致。其生成器由两个结构相同但参数独立的编码器和一个解码器组成：第一编码器将输入 x 压缩为潜变量 z ，解码器由 z 生成重建图 $\hat{x}$ ，第二编码器再将 $\hat{x}$ 编码为 $\hat{z}$ 。

判别器是 GANomaly 的另一组成部分。训练时，它同时接收真实正常图 x 和重建图 $\hat{x}$ ，学习把前者判断为 1、后者判断为 0。判别器还返回分类头之前的内部特征 $\phi_D(\cdot)$ ：生成器不仅要降低像素重建误差和潜变量误差，还要使 $\phi_D(\hat{x})$ 接近 $\phi_D(x)$ 。因此，判别器相当于不断学习真实图与重建图差异的检查器，为生成器提供比单纯像素比较更丰富的训练信号。

在一个 batch 内，代码先固定生成器产生的重建图并更新一次判别器，再冻结判别器参数、重新前向计算生成器并更新一次生成器。测试阶段不再更新任何参数，也不直接使用判别器的真假概率，而是以 z 与 $\hat{z}$ 的平均绝对差作为异常分数。

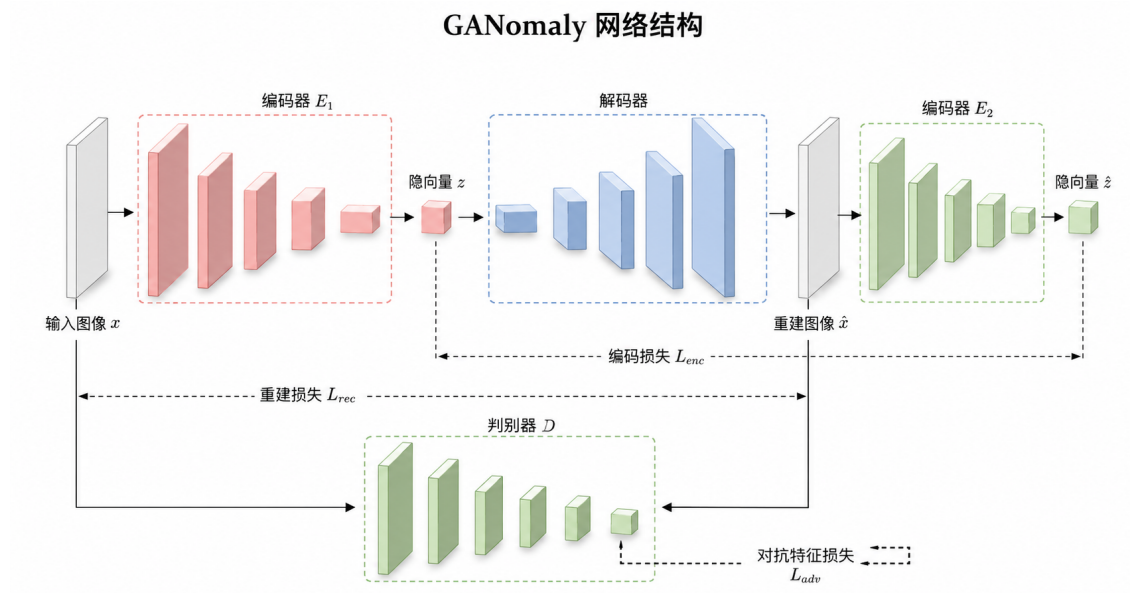


图 1 GANomaly 网络结构示意图

表 6 GANomaly 模型说明

问题	说明
输入与输出	输入为 $[B, 3, 64, 64]$ ；生成器输出同尺寸重建图，两个编码器分别输出 $[B, 128, 1, 1]$ 的 z 与 $\hat{z}$
关键模块	编码器一把原图压缩为 z ，解码器由 z 重建 $\hat{x}$ ，参数独立但结构相同的编码器二把 $\hat{x}$ 编为 $\hat{z}$ ；判别器同时输出真假概率与中间特征
训练信号	生成器使用 L1 重建损失、潜变量 MSE 和判别器特征 MSE，权重为 40 : 1 : 1；判别器使用真实图为 1、重建图为 0 的 BCE
异常分数	每块采用 z 与 $\hat{z}$ 的平均绝对差；整图同时报告四块分数的最大值与平均值
自己的理解	模型只用正常图训练，因而正常纹理应能稳定经历“编码-重建-再编码”；异常纹理若偏离正常分布，通常会使两个潜变量不一致，从而得到更高分数

3.1.2 训练目标与异常评分

本项目 GANomaly 基线的三个生成器损失分别为

$$\mathcal{L}_{\text{rec}} = \text{mean}(|x - \hat{x}|), \quad (1)$$

$$\mathcal{L}_{\text{enc}} = \text{mean}((z - \hat{z})^2), \quad (2)$$

$$\mathcal{L}_{\text{adv}} = \text{mean}([\phi_D(\hat{x}) - \text{stopgrad}(\phi_D(x))]^2). \quad (3)$$

其中，重建项使用 L1 均值，编码项和判别器特征项使用 MSE。生成器总损失为

$$\mathcal{L}_G = 40\mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{enc}} + \mathcal{L}_{\text{adv}}. \quad (4)$$

判别器使用二元交叉熵：

$$\mathcal{L}_D = \frac{1}{2} [\text{BCE}(D(x), 1) + \text{BCE}(D(\hat{x}), 0)]. \quad (5)$$

测试时每个裁剪块的异常分数为

$$A_{\text{GAN}}(x) = \text{mean} |z - \hat{z}|. \quad (6)$$

一张原图的四个裁剪块分别得到 $A_1, \dots, A_4$ ，本项目固定同时报告

$$A_{\text{max}} = \max_i A_i, \quad A_{\text{mean}} = \frac{1}{4} \sum_{i=1}^4 A_i. \quad (7)$$

两种聚合产生两套整图排序，因此各自计算出的 AUROC 可能不同。

3.2 MF-GANomaly

3.2.1 基本原理

MF-GANomaly 是在 GANomaly 基础上提出的多特征异常检测方法。原版 GANomaly 最终主要利用高度压缩的潜变量 z 与 $\hat{z}$ 判断异常，这种深层表示善于概括整体结构，却可能在压缩过程中弱化断丝、弯曲和局部纹理错位等细小异常。MF-GANomaly 因此保留原有的“编码-重建-再次编码”主流程，同时增加原图与重建图在像素层面和浅层特征层面的比较，使模型兼顾整体语义与局部纹理。

具体而言，第一编码器从输入图 x 得到潜变量 z 和第一层浅层特征 $f_1(x)$ ，解码器根据 z 生成重建图 $\hat{x}$ ，第二编码器再从 $\hat{x}$ 得到 $\hat{z}$ 和 $f_1(\hat{x})$ 。两个编码器结构相同但参数相互独立。像素差异反映图像表面的重建偏差，浅层特征差异重点反映边缘、线条和周期纹理变化，深层编码差异则描述整体表示是否一致；三类信息互相补充。本项目采用论文式编码器变体：基线已有的中间卷积层继续使用 Batch Normalization (BN) 和 LeakyReLU，并在第一层与最后一层卷积后也加入 BN 和 LeakyReLU，使各层特征的数值分布在训练中更稳定。

判别器的作用与 GANomaly 基线相同。它使用真实正常图作为真样本、生成器重建图作为假样本，通过二元交叉熵学习区分二者；其分类头之前的内部特征还用于生成器的特征匹配损失，促使重建图在判别器看来更接近真实正常图。训练时，生成

器综合优化像素距离、浅层特征距离、潜变量距离和判别器特征距离，判别器与生成器仍按 batch 交替更新。

需要区分训练损失和测试异常分数：训练损失用于反向传播并修改网络参数，而测试分数只在模型训练完成后衡量样本偏离正常模式的程度，不参与梯度计算。本项目的测试分数直接合并像素、浅层特征和深层编码三种差异；分数越大，表示输入越难被模型按照正常纹理规律稳定重建与编码，因此越可能是异常图像。

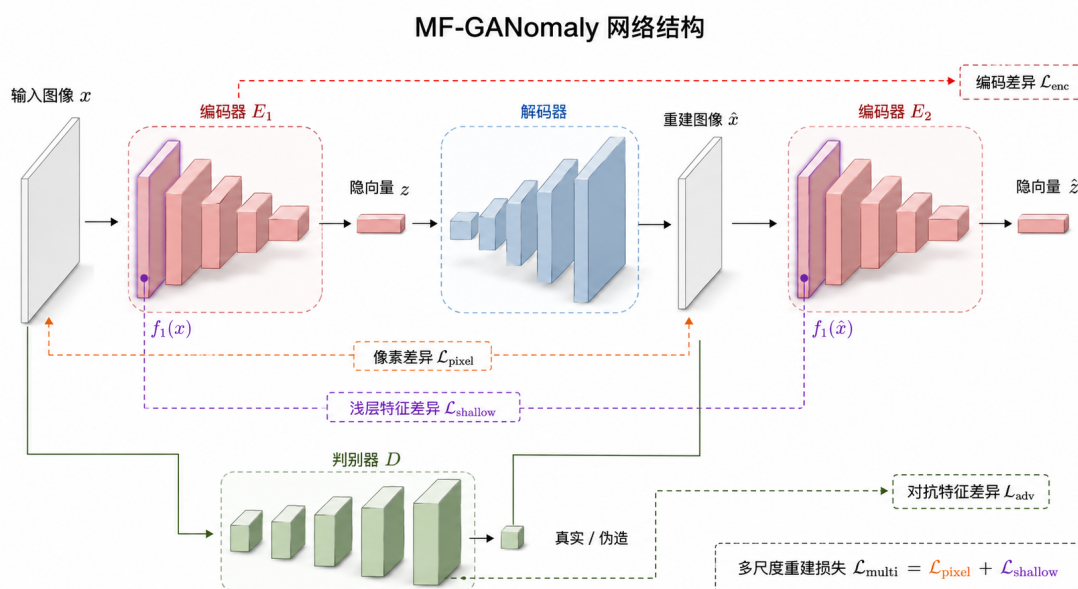


图 2 MF-GANomaly 网络结构示意图

表 7 MF-GANomaly 相对基线的改动

问题	说明
改动位置	两个编码器除返回最终潜变量外，还返回第一层卷积后的 $[B, 64, 32, 32]$ 浅层特征
新增损失	将原图与重建图的像素 L2 距离、两侧第一层浅层特征 L2 距离相加，构成多特征重建项
归一化与激活	编码器各卷积层后采用 BN 与 LeakyReLU；相较基线，本项目的论文式变体还为第一层和最后一层补充这两个操作
预期作用	深层潜变量偏向整体语义，第一层仍保留边缘、周期纹理和局部细节，因此对断丝、弯曲等小缺陷更敏感
实现差异	论文对“多尺度”的具体取层和跨块聚合未完全公开；本项目明确使用第一层浅特征，按逐样本 L2 范数实现，并同时报告最大值与平均值聚合

3.2.2 关键公式

MF-GANomaly 在像素重建距离之外加入两个编码器第一层浅特征的距离：

$$\mathcal{L}_{\text{shallow}} = \|f_1(x) - f_1(\hat{x})\|_2, \quad \mathcal{L}_{\text{con}} = \|x - \hat{x}\|_2 + \mathcal{L}_{\text{shallow}}. \quad (8)$$

训练时仍采用重建项、编码项和判别器特征项的 40 : 1 : 1 加权组合；本项目测试分数为

$$A_{\text{MF}}(x) = \|z - \hat{z}\|_2 + 40 (\|x - \hat{x}\|_2 + \|f_1(x) - f_1(\hat{x})\|_2). \quad (9)$$

编码器中的 BN 对每个特征通道分别计算一个 mini-batch 内（同时包含该通道各空间位置）的均值与方差。设该通道参与统计的元素数为 m ，输入为 $u_1, \dots, u_m$ ，则

$$\mu_B = \frac{1}{m} \sum_{i=1}^m u_i, \quad \sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (u_i - \mu_B)^2, \quad (10)$$

$$\hat{u}_i = \frac{u_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{u}_i + \beta. \quad (11)$$

其中， ϵ 防止分母为零， γ 与 β 是网络通过训练学习的缩放和偏移参数。BN 先把特征调整到较稳定的尺度，再允许网络利用 γ 和 β 学回合适的分布；它本身不是异常分数，也不会单独判断异常，而是帮助梯度传播和训练稳定。BN 后使用 LeakyReLU：

$$\text{LeakyReLU}(a) = \begin{cases} a, & a \geq 0, \\ 0.2a, & a < 0, \end{cases} \quad (12)$$

从而在提供非线性的同时保留少量负半轴信息，降低神经元长期没有梯度的风险。

3.3 Mask Encoders

3.3.1 基本原理

GANomaly 类方法在训练时把完整正常图同时作为输入和重建目标，模型可能逐渐学会近似复制输入。若测试时也能把异常区域一并重建，原图与重建图之间的差异就会减小，从而造成漏检。Mask Encoders 通过人为遮挡正常图的一部分构造自监督任务：输入是中心区域缺失的正常图，监督目标则是遮挡前的完整正常图。由于缺失位置没有可供直接复制的像素，模型必须利用周围纹理和结构推断中心内容，从而学习正常 Grid 的排列规律，而无需人工标注异常位置。

模型采用单编码器-解码器生成器。编码器把 $[B, 3, 128, 128]$ 的输入压缩为 $[B, 512, 1, 1]$ 的潜变量，解码器再将其恢复为同尺寸重建图 $\hat{x}$ 。与 GANomaly 和 MF-GANomaly

不同，该模型没有第二编码器，不计算 z 与 $\hat{z}$ 的编码损失，而是把学习重点放在遮挡补全和图像重建质量上。

训练分为两个阶段。第一阶段固定抽取 50% 的正常裁剪块，将中心 64×64 区域遮挡，并以完整图为目标训练 80 轮；第二阶段继承第一阶段生成器和判别器权重，重新创建 Adam 优化器，再用全部完整正常裁剪块微调 80 轮。前一阶段学习根据上下文补全正常纹理，后一阶段把这种能力适配到测试时使用的完整图重建任务。

判别器把输入图逐层压缩为 100 维特征，再通过线性层和 Sigmoid 输出真假概率。它以完整正常图为真样本、生成器重建图为假样本进行训练；生成器则直接要求 $D(\hat{x})$ 接近 1。因此，判别器为生成器提供“重建图是否像真实正常图”的学习信号，补充单纯 L2 像素距离可能产生的模糊结果。两个网络在每个 batch 内交替更新。测试阶段不再更新参数，也不使用判别器概率作为异常分数，而是用原图与重建图的 L2 距离衡量异常程度。

Mask Encoders 网络结构

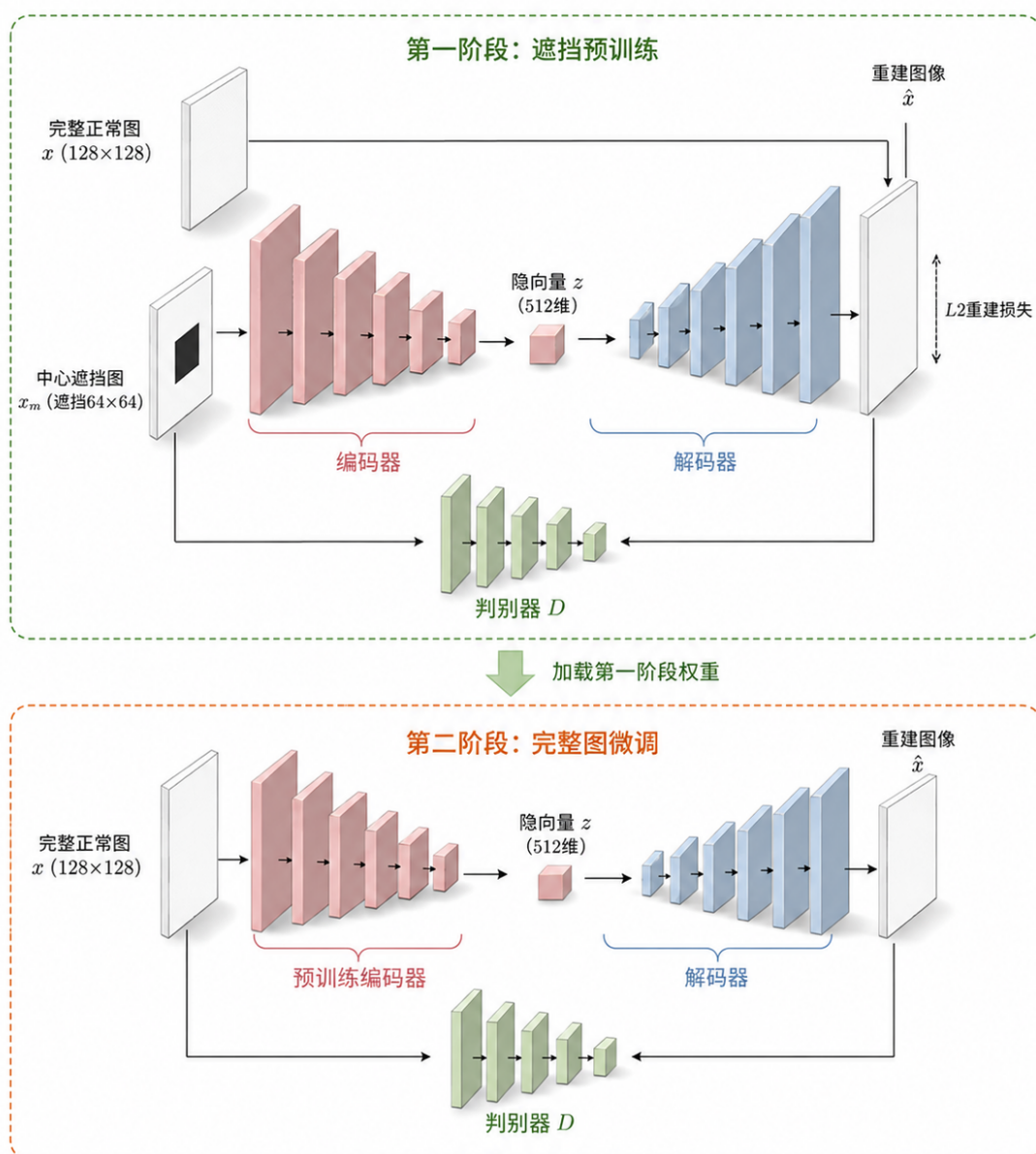


图 3 Mask Encoders 两阶段训练结构示意图

表 8 Mask Encoders 两阶段训练

阶段	说明
第一阶段	固定抽取 50% 的正常裁剪块，在 128×128 输入中心遮挡 64×64 区域，以未遮挡完整图为目标训练 80 轮
第二阶段	载入第一阶段生成器与判别器权重，重新创建 Adam 优化器，用全部完整正常裁剪块微调 80 轮
预期作用	模型无法从遮挡区域直接复制像素，只能利用周围正常纹理推断缺失内容，因此被迫学习 Grid 的结构规律；第二阶段再适配完整图重建任务
测试评分	每块异常分数为原图与重建图的 L2 距离；额外统计裁剪块 PSNR 与 SSIM，整图 AUROC 同时使用四块最大值和平均值

3.3.2 关键公式

第一阶段的遮挡输入可概括为

$$x_{\text{mask}} = x \odot M. \quad (13)$$

概念上 $M = 0$ 表示遮挡位置、 $M = 1$ 表示保留位置；实际代码因输入已归一化到 $[-1, 1]$ ，直接把中心 64×64 区域填为 -1 。无论输入是第一阶段的遮挡图还是第二阶段的完整图，监督目标 x 均为完整正常图。每个样本的 L2 重建距离为

$$d_2(x, \hat{x}) = \|x - \hat{x}\|_2 = \sqrt{\sum_j (x_j - \hat{x}_j)^2}, \quad (14)$$

其中 j 遍历一张裁剪图的全部通道和空间位置。生成器的重建项、对抗项与总损失分别为

$$\mathcal{L}_{\text{rec}} = \text{mean}_{x \in B} d_2(x, \hat{x}), \quad (15)$$

$$\mathcal{L}_{\text{adv}} = \text{BCE}(D(\hat{x}), 1), \quad (16)$$

$$\mathcal{L}_G^{\text{Mask}} = 40\mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{adv}}. \quad (17)$$

其中，对抗项的目标标签为 1，表示生成器希望判别器把重建图判断为真实图。判别器损失为

$$\mathcal{L}_D^{\text{Mask}} = \frac{1}{2} [\text{BCE}(D(x), 1) + \text{BCE}(D(\text{stopgrad}(\hat{x})), 0)]. \quad (18)$$

更新判别器时对 $\hat{x}$ 使用 stopgrad，使该步骤只修改判别器参数。测试时每个裁剪块的异常分数为

$$A_{\text{Mask}}(x) = d_2(x, \hat{x}) = \|x - \hat{x}\|_2. \quad (19)$$

异常图若不能被模型按照正常纹理规律稳定重建，通常会得到更大的距离和异常分数。

3.4 TS-GAN

3.4.1 基本原理

前三种方法只利用正常图片训练，能够学习正常模式，却没有使用现实中可能已经收集到的少量异常样本。TS-GAN采用弱监督双流结构，同时建立正常生成器 G_N 和异常生成器 G_A ：前者学习正常图的重建规律，后者利用少量已知异常图学习异常模式。两个生成器结构相同但参数完全独立，避免第二阶段的异常训练直接改变正常生成器，使正常支路仍能作为稳定参照。

两条生成器都由编码器和解码器构成，并加入一条跳跃连接。编码器把 $[B, 3, 64, 64]$ 的输入压缩成 $[B, 512, 1, 1]$ 的潜变量；编码器第 4 层的 $[B, 512, 4, 4]$ 特征与解码器第一层的同尺寸输出沿通道维拼接，得到 $[B, 1024, 4, 4]$ ，再继续解码。潜变量提供高度压缩的整体信息，跳跃特征则向解码端补充局部空间结构和纹理细节。

训练分为两个阶段。第一阶段用正常裁剪块训练 G_N 与共享判别器 D 共 150 轮；此时 G_A 保持冻结。第二阶段冻结已经训练好的 G_N ，从 Grid 五类缺陷中各取 2 张、合计 10 张异常原图训练 G_A 与同一个 D 共 50 轮。共享判别器继承第一阶段模型参数，但第二阶段重新建立 Adam 优化器。判别器始终判断“真实输入还是生成器重建图”，并不直接分类正常与异常：第二阶段的真实异常图标签仍为 1，因为这里的 1 表示真实图片。

测试时，同一输入分别经过 G_N 和 G_A ，得到正常支路差异 S_N 与异常支路差异 S_A 。正常图通常容易被 G_N 重建而得到较小的 S_N ；异常图通常更难被 G_N 重建，同时更接近 G_A 学过的异常模式，因此 S_N 较大而 S_A 较小。最终使用 $S_N - 0.1S_A$ 排序，使异常支路成为正常支路的对照参照。

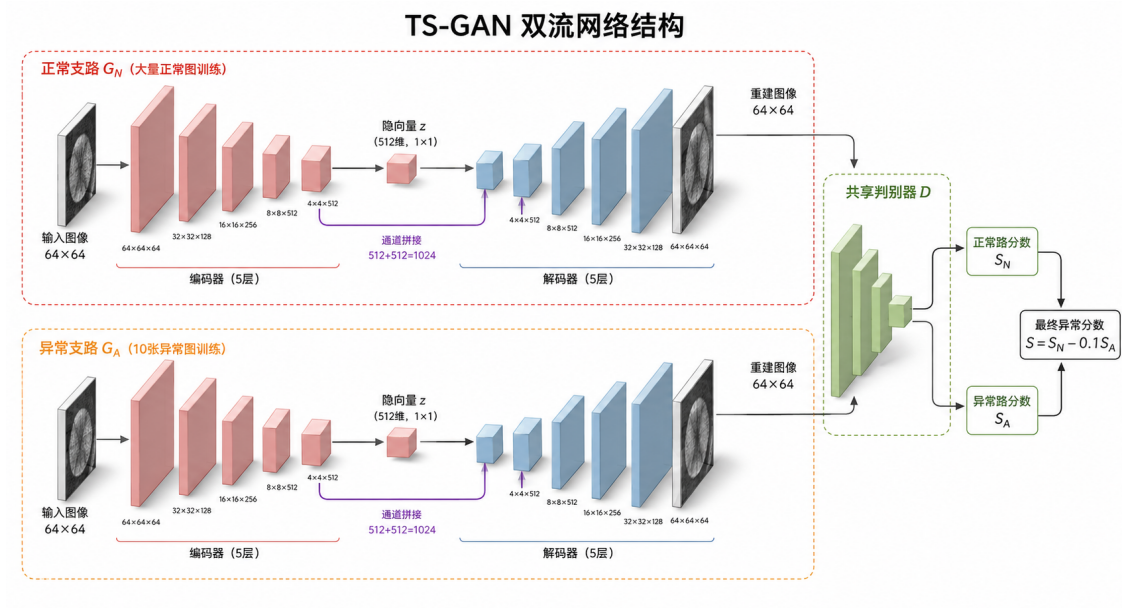


图 4 TS-GAN 双流结构示意图

表 9 TS-GAN 双流模型说明

问题	说明
正常支路	第一阶段只用正常裁剪块训练正常生成器与共享判别器 150 轮, 生成器损失含重建、判别器特征和对抗项
异常支路	第二阶段冻结正常生成器; 按种子 42 从五类缺陷各选 2 张异常图, 共 10 张, 训练异常生成器与同一判别器 50 轮
共享部分	两个生成器参数互不共享; 同一判别器在不同阶段分别接收真实输入和对应支路重建图, 其特征既参与生成器损失也参与测试评分
跳跃连接	编码器第 4 层 $[B, 512, 4, 4]$ 与解码器第 1 层输出 $[B, 512, 4, 4]$ 沿通道维拼接为 $[B, 1024, 4, 4]$, 向解码端补充空间细节
训练权重	正常与异常生成器训练时均采用重建、判别器特征、直接对抗三项损失, 权重为 $40 : 1 : 1$; 两阶段学习率分别为 0.0002 和 0.0001
最终分数	正常支路分数 S_N 减去 0.1 倍异常支路分数 S_A ; 若输入更像异常支路学过的异常模式, S_A 较小, 最终异常分数仍相对升高

3.4.2 关键公式

设当前训练支路为 $b \in \{N, A\}$ ，其生成器输出为 $\hat{x}_b$ 。两条支路训练时使用相同形式和相同权重的生成器损失：

$$\mathcal{L}_{\text{rec}}^{(b)} = \text{mean}_{x \in B} \|x - \hat{x}_b\|_2, \quad (20)$$

$$\mathcal{L}_{\text{fea}}^{(b)} = \text{mean}_{x \in B} \|\text{stopgrad}(\phi_D(x)) - \phi_D(\hat{x}_b)\|_2, \quad (21)$$

$$\mathcal{L}_{\text{adv}}^{(b)} = \text{BCE}(D(\hat{x}_b), 1), \quad (22)$$

$$\mathcal{L}_{G_b} = 40\mathcal{L}_{\text{rec}}^{(b)} + \mathcal{L}_{\text{fea}}^{(b)} + \mathcal{L}_{\text{adv}}^{(b)}. \quad (23)$$

这里必须区分训练权重和测试权重：异常生成器训练时的重建权重也是 40，而不是 10。判别器在当前支路上的损失为

$$\mathcal{L}_D^{(b)} = \frac{1}{2} [\text{BCE}(D(x), 1) + \text{BCE}(D(\text{stopgrad}(\hat{x}_b)), 0)]. \quad (24)$$

测试阶段不再使用 BCE，而是分别计算正常支路和异常支路分数：

$$S_N(x) = 40\|x - \hat{x}_N\|_2 + \|\phi_D(x) - \phi_D(\hat{x}_N)\|_2, \quad (25)$$

$$S_A(x) = 10\|x - \hat{x}_A\|_2 + \|\phi_D(x) - \phi_D(\hat{x}_A)\|_2. \quad (26)$$

最终异常分数为

$$S(x) = S_N(x) - \eta S_A(x), \quad \eta = 0.1. \quad (27)$$

其中，测试参数 $\alpha = 40$ 、 $\beta = 10$ 分别控制两条支路中像素距离相对判别器特征距离的权重， $\eta = 0.1$ 控制异常支路对最终结果的影响。这三个数用于测试评分组合，不会反向传播修改模型参数。

3.4.3 论文式像素定位方法

图片级异常分数用于判断整张图片是否异常，像素定位则采用论文单独描述的滑动窗口 PSNR 方法。对于 Grid 测试图，首先只使用正常生成器得到重建图 $\hat{x}_N = G_N(x)$ 。设原图与重建图共有 n 个像素值，像素范围为 $[0, 1]$ ，则均方误差和峰值信噪比分别为

$$\text{MSE}(x, \hat{x}_N) = \frac{1}{n} \sum_{i=1}^n (x_i - \hat{x}_{N,i})^2, \quad (28)$$

$$\text{PSNR}(x, \hat{x}_N) = 10 \log_{10} \left(\frac{1}{\text{MSE}(x, \hat{x}_N)} \right). \quad (29)$$

PSNR 越高，表示两张图越相似。先计算整个 64×64 裁剪块的整体 PSNR，记为 P_{global} ；再用窗口遍历裁剪块，在位置 (i, j) 计算局部 PSNR $P_{\text{local}}(i, j)$ 。异常响应定义为

$$H(i, j) = \max(P_{\text{global}} - P_{\text{local}}(i, j), 0). \quad (30)$$

若某个局部区域比整块图的平均水平重建得更差，其局部 PSNR 会降低， $H(i, j)$ 因而增大。所有响应最终按单张图片归一化到 $[0, 1]$ 并绘制为连续热图。论文没有给出二值化阈值，也没有公开窗口大小、步长和重叠窗口的汇总方式；本项目采用窗口边长 8、步长 1，并将覆盖同一像素的窗口响应取平均。这些参数属于明确记录的复现假设，不作为论文原始设定。

3.5 四个模型的关系

表 10 四个模型对比

模型	正常数据	异常数据	训练阶段	主要改进
GANomaly	是	否	一阶段	双编码器重建基线
MF-GANomaly	是	否	一阶段	增加浅层多特征约束
Mask Encoders	是	否	两阶段	遮挡补全预训练与完整图微调
TS-GAN	是	少量	两阶段	正常和异常双流生成器与共享判别器

4 统一符号与评价指标

4.1 符号定义

表 11 关键符号说明

符号	名称或形状	含义
x	$3 \times 64 \times 64$ 或 $3 \times 128 \times 128$	输入图像裁剪块
$\hat{x}$	同 x	生成器重建的图像
z	GANomaly/MF 为 $128 \times 1 \times 1$ ； TS-GAN 为 $512 \times 1 \times 1$	第一编码器提取的潜变量

符号	名称或形状	含义
$\hat{z}$	同 z	第二编码器对重建图的编码
$D(\cdot)$	概率与特征	真假概率用于 BCE；分类头之前的特征用于特征匹配
$f_l(\cdot)$	第 l 层特征	MF-GANomaly 使用的浅层特征
M	掩码	Mask Encoders 使用的二值遮挡图
S_N, S_A	标量	TS-GAN 正常支路与异常支路分数
$A_{\max}, A_{\text{mean}}$	标量	四个裁剪块分数的最大值与平均值

4.2 评价指标

$$\text{TPR} = \frac{\text{TP}}{\text{TP} + \text{FN}}, \quad \text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}. \quad (31)$$

ROC 曲线遍历全部可能阈值，横轴为 FPR，纵轴为 TPR；AUROC 是曲线下面积。直观上，它等于随机抽取一张异常图和一张正常图时，模型把异常图分数排得更高的概率。AUROC 不需要先固定分类阈值，0.5 约等于随机排序，越接近 1 越好。本报告现有主结果是图片级 AUROC；Mask Encoders 的 PSNR、SSIM 是裁剪块级计算后取均值。独立验证若报告 F1、精确率或召回率，必须先说明阈值来源。

5 实现过程与训练设置

5.1 项目结构

表 12 关键代码模块

文件或目录	职责	对应模型
src/anomaly_reproduction/models.py	定义双编码器生成器和输出概率、特征图的判别器	GANomaly
src/anomaly_reproduction/models_mf.py	复用基线层定义，额外返回两个编码器的第一层浅特征	MF-GANomaly
src/anomaly_reproduction/models_mask.py	定义 128×128 的补全生成器和判别器	Mask Encoders
src/anomaly_reproduction/models_ts.py	定义两套独立生成器、单跳跃连接和共享判别器	TS-GAN
src/anomaly_reproduction/losses*.py	实现各模型生成器损失、判别器损失与异常评分组件	全部模型
src/anomaly_reproduction/evaluate*.py	生成裁剪块分数、整图聚合结果和 AUROC	全部模型

5.2 统一超参数

表 13 训练超参数

参数	GANomaly	MF	Mask	TS-GAN
输入尺寸	64×64	64×64	128×128	64×64
批大小	16	16	16	16
训练轮数	150	150	80 + 80	150 + 50
学习率	0.0002	0.0002	0.0002	0.0002/0.0001
优化器	Adam	Adam	Adam	Adam
随机种子	42	42	42	42

5.3 训练流程

1. 数据集只读取 Grid 正常训练图，固定四块裁剪、缩放并归一化；DataLoader 以 batch 16 打乱训练样本。
2. 每个 batch 先用真实图和生成器重建图计算判别器 BCE，执行 zero_grad、反向传播和判别器 Adam 更新；重建图在此步骤中与生成器计算图分离。

3. 随后冻结判别器参数但保留其对输入的梯度，重新前向计算生成器损失，反向传播并更新生成器。两种网络因此在同一 batch 内交替学习。
4. 每轮记录各损失,按固定间隔保存检查点,并在训练结束保存 `checkpoint_final.pt`、历史文件和配置副本。
5. 测试时每个裁剪块独立评分；按原始图片路径将四块重新分组，分别取最大值和平均值，再与图片级标签计算 AUROC。

5.4 与论文不一致或未公开的实现细节

表 14 复现假设与影响

序号	论文描述	本项目选择	可能影响
1	四块分数聚合方式未明确	同时报告最大值与平均值	不同聚合可能改变图片级排序
2	TS-GAN异常训练图名单未公开	每类随机抽取 2 张，种子 42	测试集组成与论文可能不同
3	MF-GANomaly 多尺度取层未完整公开	使用两个编码器第一层卷积特征，距离采用逐样本 L2	可能改变损失量级和对局部纹理的敏感度
4	Mask Encoders 判别器分类头细节未完全公开	将 100 维分类特征经线性层与 Sigmoid 变为单一概率	影响对抗训练强度，但保持真假二分类目标
5	论文环境为 PyTorch 1.8/CUDA 11.4/RTX 2060	本项目使用 PyTorch 2.11/CUDA 12.8/RTX 5070 Laptop	算子实现、随机性和训练速度可能不同

6 MVTEC Grid 主实验结果

6.1 总体评价问题与对照关系

表 15 实验问题

序号	要回答的问题	对照或证据
1	四个模型是否复现出论文所述趋势	与论文报告值及基线比较
2	模型是否能识别未参与训练的数据	独立验证集 AUROC 与分类案例
3	模型在哪些缺陷上容易失败	按缺陷类别统计并分析误报、漏报
4	四块最大值和平均值哪种更合适	在预先固定的验证协议下比较
5	TS-GAN 能否定位 Grid 缺陷区域	论文式滑窗 PSNR 热图、Pixel AUROC 与案例分析

6.2 Grid 首轮复现结果

表 16 MVTec AD Grid 图片级结果

模型	测试图片	最大值 AUROC	平均值 AUROC	论文值
GANomaly	78	0.6817	0.7026	0.658
MF-GANomaly	78	0.7703	0.7510	0.740
Mask Encoders	78	0.9365	0.9056	0.785
TS-GAN	68	0.9534	0.9240	0.806

必要说明：TS-GAN 使用 10 张异常图进行弱监督训练，并在测试时排除了这些图片，因此其测试集与其他三个模型不同。以上结果来自单个随机种子，不能直接作为严格的模型排名。

表 17 Mask Encoders 重建质量补充结果

指标	本项目	论文值
裁剪块 PSNR 均值	24.2551	23.631
裁剪块 SSIM 均值	0.7862	0.802

从本轮结果看，MF-GANomaly 相比 GANomaly 的最大值 AUROC 提高 0.0886，说明浅层纹理约束在 Grid 上有效。Mask Encoders 与 TS-GAN 的数值明显高于本项目基线和论文表格值，但更高数值同时受到固定四块裁剪、实现细节和测试集组成影响，只能视为值得进一步验证的现象。GANomaly 在平均聚合下更高，其余三种方法在最大聚合下更高，说明聚合方法改变的是整图排序关系，而不是“平均值不应超过最大值”的数值常识：表中比较的是两套分数各自计算得到的 AUROC。

6.3 TS-GAN 滑动窗口 PSNR 像素定位实验

本实验读取 TS-GAN 第 50 轮异常阶段检查点，不重新训练模型；测试仍排除异常支路训练见过的 10 张异常图，共使用 21 张正常图和 47 张异常图。每个 64×64 裁剪块由正常支路重建，采用窗口边长 8、步长 1 计算滑动窗口 PSNR 响应，随后将四块热图拼接为 128×128 的展示结果。论文只展示连续异常热图，没有给出像素二值化阈值，因此本实验不生成预测掩码，也不报告依赖固定阈值的 Precision、Recall、F1 或 IoU。

表 18 TS-GAN 论文式 PSNR 热图的额外像素级评估

评估范围	Pixel AUROC
全部 68 张测试图	0.9683
弯曲 (Bent)	0.9855
断裂 (Broken)	0.9888
胶水附着 (Glue)	0.9343
金属杂质 (Metal contamination)	0.9958
毛线缠绕 (Thread)	0.9673

指标说明：论文没有报告上述 Pixel AUROC，0.9683 是本项目在明确滑窗参数下增加的评价结果，不能写成论文复现值。它表示随机抽取一个真实异常像素和一个正常像素时，连续热图把异常像素排在更高位置的概率约为 96.83%，并不表示 96.83% 的像素都被正确分类。

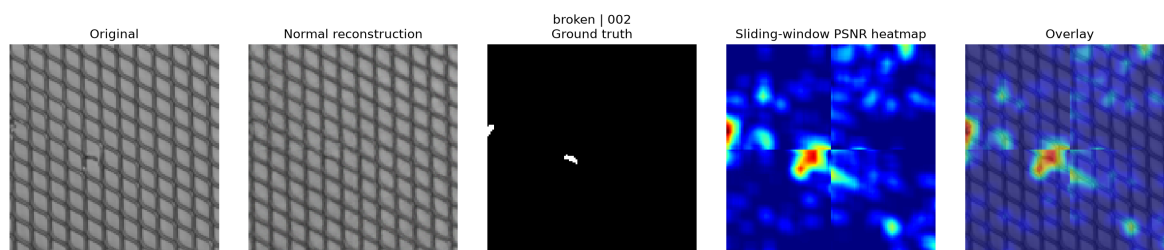


图 5 TS-GAN 在断裂样本上的滑窗 PSNR 热图示例。两个真实缺陷位置均出现集中高响应，但裁剪边界和少量正常纹理仍存在响应。

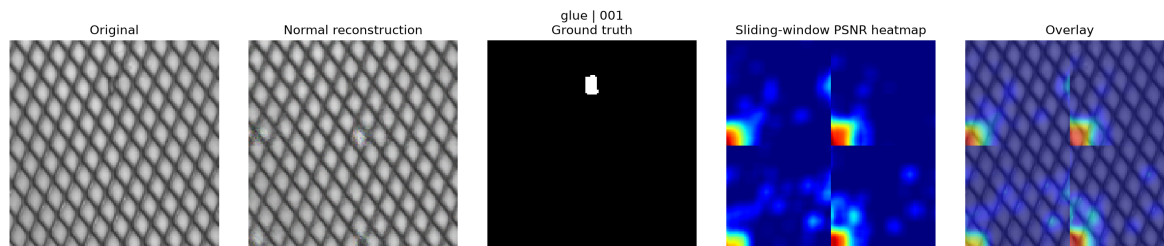


图 6 TS-GAN 在胶水附着样本上的较差案例。该图单图 Pixel AUROC 为 0.6628，说明小尺寸、低对比度缺陷仍可能被纹理重建误差掩盖。

总体结果明显好于此前采用逐像素双支路残差得到的噪声热图，说明论文采用局部窗口比较而非单像素比较是关键：窗口能够聚合邻域结构误差，而“整体 PSNR 减局部 PSNR”进一步突出相对整图平均水平重建明显较差的区域。不过，正常样本的纹理和四块拼接边界仍可能出现局部高响应；后续应在独立数据上验证，并进行窗口大小与步长的敏感性实验。

7 扩展实验一：AITEK 零样本外部验证

7.1 实验目的与数据协议

本实验回答的问题是：只在 MVTEC AD Grid 上训练的 TS-GAN，能否不经重新训练而直接识别另一数据集中的织物缺陷？因此，本章读取 Grid 实验已经训练完成的第 50 轮异常阶段检查点，模型参数保持冻结，AITEK 异常图既不参与训练，也不参与模型选择或阈值确定。这是一项跨数据集、跨纹理域的零样本外部验证，用于评价模型的直接迁移能力。

AITEK 共使用 35 张正常图作为阈值校准集，另以 106 张正常图和 106 张异常图作为图片级测试集。阈值仅取正常校准分数的 0.99 分位数；像素评价使用 106 张正常图和 105 张带有效掩码的异常图，排除缺少掩码的 0100_025_08.png。本章结果不能解释为“AITEK 训练效果”，因为模型从未使用 AITEK 图像更新参数。

7.2 图片级结果

表 19 TS-GAN 在 AITEK-AFID 上的零样本图片级结果

汇总方式	AUROC	AP	准确率	F1	召回率	阈值
图块最大值	0.4463	0.4856	0.4953	0.0531	0.0283	2361.30
图块平均值	0.5482	0.5392	0.5000	0.0536	0.0283	882.15

两种阈值均只由 35 张正常校准图的 0.99 分位数得到。最大值汇总的混淆矩阵为 TN=102、FP=4、FN=103、TP=3，平均值汇总为 TN=103、FP=3、FN=103、TP=3。虽然误报较少，但原因是阈值较高，代价是 106 张异常图中仅检出 3 张；因此约 0.50 的准确率不能解释为模型效果良好。与 Grid 上 0.9534（最大汇总）和 0.9240（平均汇总）的图片级 AUROC 相比，AITEK 分别下降 0.5071 和 0.3758，说明整图异常分数没有成功迁移到新的织物域。

7.3 像素级总体与分类别结果

表 20 AITEX-AFID 像素级总体结果

评估范围	Pixel AUROC	Pixel AP
全部 211 张有像素评价资格的图像	0.7472	0.0060
仅 105 张有掩码的异常图像	0.7475	0.0114

Pixel AUROC 表示随机抽取一个异常像素和一个正常像素时，热图把异常像素排在更高位置的概率。AITEX 的 0.7472 比 Grid 的 0.9683 下降 0.2211，说明定位排序能力仍部分保留，但跨域后明显退化。Pixel AP 对稀少的异常像素更敏感；0.0060 表明热图虽然常在缺陷附近产生较高响应，却同时覆盖大量正常区域，精确定位能力较弱。Grid 实验未计算 Pixel AP，因此不能对该指标作直接数值比较。

表 21 AITEX-AFID 按缺陷类型的像素级结果

代码	官方缺陷名称（中文释义）	图像数	Pixel AUROC	Pixel AP
002	Broken end（断经）	9	0.8170	0.005645
006	Broken yarn（断纱）	8	0.8588	0.008655
010	Broken pick（断纬）	10	0.6329	0.029709
016	Weft curling（纬纱卷曲）	3	0.8887	0.007636
019	Fuzzyball（毛球）	39	0.6773	0.000050
022	Cut selvage（切边缺陷）	9	0.5851	0.000470
023	Crease（折痕）	5	0.7563	0.002407
025	Warp ball（经纱结团）	5	0.6641	0.000046
027	Knots（结头）	1	0.8855	0.001299
029	Contamination（污染）	1	0.7145	0.000497
030	Nep（棉结/纤维结粒）	14	0.7753	0.000597
036	Weft crack（纬向裂缝）	1	0.8235	0.468473

其中 002、030 等数字是 AITEX 文件名中的官方缺陷类型代码，不是模型分数。036 仅有 1 张图，其较高 AP 不具有稳定的类别统计意义；027、029 同样只有 1 张图，不能据此判断模型对这些缺陷具有稳定优势。

7.4 表现较好的案例

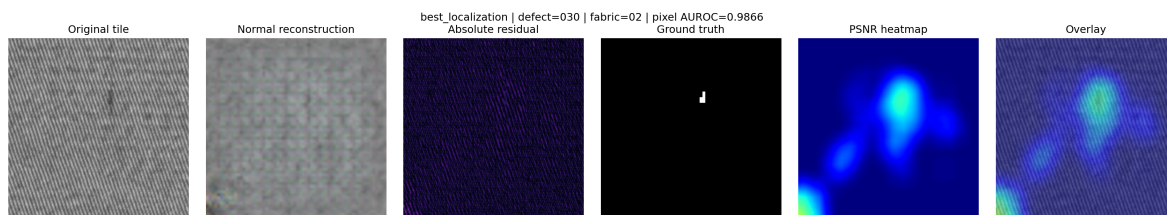


图 7 AITEX 棉结缺陷 0086_030_02 的较好定位案例，单图 Pixel AUROC 为 0.9866。热图高响应覆盖真实缺陷附近，说明 Grid 上学习到的局部重建差异对部分跨域缺陷仍然有效；但 Pixel AP 仅为 0.0039，提示响应范围仍远大于真实掩码。

7.5 表现不理想的案例

7.5.1 误报案例

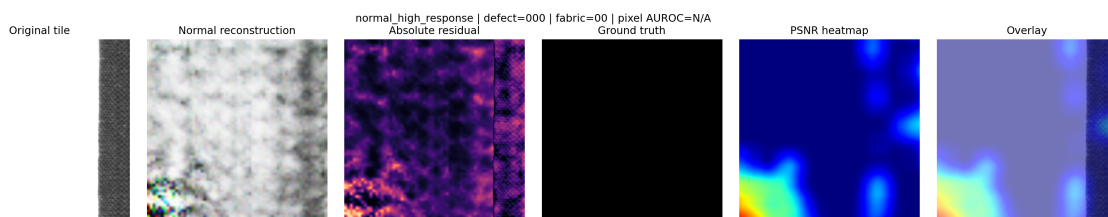


图 8 AITEX 正常样本 0008_000_00 的高响应案例。真实掩码为空，但热图仍在正常纹理区域产生明显响应，说明材质变化和周期纹理的重建误差会被误当作缺陷证据。

7.5.2 漏报案例

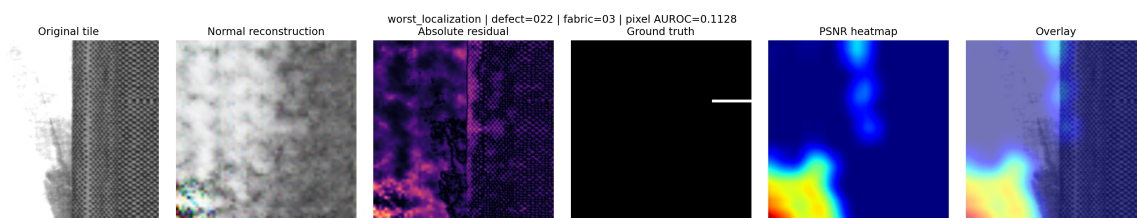


图 9 AITEX 切边缺陷 0104_022_03 的较差定位案例，单图 Pixel AUROC 仅为 0.1128。热图主要响应正常纹理而非真实缺陷，反映 64×64 输入分辨率、材质域偏移及单尺度窗口共同造成的定位失败。

7.6 零样本测试结果小结

AITEX 独立验证不支持“Grid 上的高性能可以直接推广到其他织物数据”的结论。图片级平均汇总 AUROC 仅为 0.5482，且在正常校准阈值下只检出 3/106 张异常图，说明异常分数的尺度和排序均发生明显变化。像素级 Pixel AUROC 为 0.7472，表明

局部 PSNR 热图仍保留部分位置排序能力；然而 Pixel AP 只有 0.0060，结合正常高响应与切边缺陷漏定位案例可知，热图存在大面积非缺陷响应。后续改进应优先考虑目标域正常样本适配、更高输入分辨率、多尺度窗口及基于正常目标域数据的热图校准，而不能使用 AITEX 异常测试标签反复选择参数。

8 扩展实验二：AITEX 域内重新训练

8.1 实验目的与数据划分

本实验回答的问题是：如果允许模型在 AITEX 目标域重新训练，TS-GAN 能否学会该数据集的织物缺陷？零样本结果较差可能来自两种不同原因：一是 TS-GAN 本身不能处理织物缺陷，二是模型只学习了 MVTEC Grid 的纹理分布，发生了明显域偏移。为区分二者，本实验保持 TS-GAN 结构和两阶段训练方式不变，改用 AITEX-AFID 的训练划分重新训练，并只在严格隔离的留出测试集上评价。该实验属于目标域内训练，不再属于独立零样本验证，因此不能把两组数字混写为同一种实验。

所有划分均以原始长图为单位完成，再把长图切成不重叠的 256×256 图块并缩放到 64×64 ，从而避免同一原图的不同图块跨越训练集与测试集。种子固定为 42；每种织物取 12 张正常图训练、4 张正常图验证，其余正常图测试，共得到 84/28/29 张正常训练、验证和测试图。对样本数大于 1 的九种缺陷类型各取 1 张异常图用于第二阶段训练，共 9 张；其余 97 张异常图用于测试。代码 027、029、036 各只有一张图，全部作为训练阶段未见缺陷保留在测试集。第一阶段训练正常生成器 150 轮，第二阶段训练异常生成器 50 轮，batch size 为 32；其余损失权重与 Grid 实验保持一致。图片级阈值只由 28 张正常验证图分数的 0.99 分位数确定，没有使用测试异常标签选阈值。

8.2 图片级结果

表 22 AITEX 零样本与域内训练的图片级结果对比

训练方式	聚合	AUROC	AP	准确率	召回率	F1
Grid 零样本	最大值	0.4463	0.4856	0.4953	0.0283	0.0531
Grid 零样本	平均值	0.5482	0.5392	0.5000	0.0283	0.0536
AITEX 域内	最大值	0.8173	0.9368	0.5556	0.4536	0.6111
AITEX 域内	平均值	0.6950	0.8859	0.3889	0.2268	0.3636

最大值聚合下，域内训练相对零样本使 AUROC 提高 0.3710、AP 提高 0.4512，召回率由 0.0283 提高到 0.4536；混淆矩阵由 $TN=102$ 、 $FP=4$ 、 $FN=103$ 、 $TP=3$ 变为

TN=26、FP=3、FN=53、TP=44。由于两次实验的测试集规模不同，上述变化不是严格配对比较，但提升幅度足以说明 Grid 到 AITEX 的域偏移是零样本失败的重要原因，TS-GAN 并非完全无法学习 AITEX 纹理。

域内结果仍不能称为高可靠检测：97 张异常测试图中仍漏掉 53 张。按缺陷类型看，006 的召回率为 1.0，002 和 030 分别为 0.6250 和 0.6154；而 016、023、025、027 的召回率均为 0。三个未见缺陷类型各只有一张测试图，其中 027 漏检、029 和 036 检出，样本量不足以估计真正的未见类型泛化能力。最大值聚合明显优于平均值，说明缺陷通常只影响长图中的少数图块，平均操作容易把局部异常稀释。

8.3 像素级结果

表 23 AITEX 零样本与域内训练的像素级总体结果

训练方式	评价范围	Pixel AUROC	Pixel AP
Grid 零样本	全部 211 张可评价图	0.7472	0.0060
Grid 零样本	仅 105 张异常图	0.7475	0.0114
AITEX 域内	全部 126 张测试图	0.5213	0.0049
AITEX 域内	仅 97 张异常图	0.5207	0.0061

与图片级结果相反，域内训练后的整体 Pixel AUROC 从 0.7472 降到 0.5213，已接近随机排序的 0.5；Pixel AP 也只有 0.0049。这并不与图片级 AUROC=0.8173 矛盾，因为两个指标使用的信号不同：图片级 TS-GAN 分数联合正常支路与异常支路，而论文式 PSNR 热图只使用正常生成器，第二阶段学到的异常支路不会直接参与像素热图。此外，每张长图的热图被单独 min-max 归一化到 $[0, 1]$ ，正常图中的相对最大误差也会被强制拉到 1，使不同图片间的像素分数缺少统一标尺。

表 24 AITEX 各缺陷类型在零样本与域内训练后的像素指标

代码	域内图数	零样本 AUROC	域内 AUROC	零样本 AP	域内 AP
002	8	0.8170	0.9308	0.005645	0.022670
006	7	0.8588	0.9649	0.008655	0.098584
010	9	0.6329	0.7843	0.029709	0.025140
016	2	0.8887	0.9592	0.007636	0.011616
019	38	0.6773	0.9576	0.000050	0.000901
022	8	0.5851	0.5059	0.000470	0.001720
023	4	0.7563	0.9882	0.002407	0.031073
025	5	0.6641	0.8420	0.000046	0.000217
027	1	0.8855	0.9993	0.001299	0.214902
029	1	0.7145	0.9980	0.000497	0.066631
030	13	0.7753	0.9804	0.000597	0.004914
036	1	0.8235	0.4687	0.468473	0.189008

分类别 AUROC 大多上升，而全体像素合并后的 AUROC 下降，看似反常，实质上

与汇总方式有关。分类指标是在该类别内部合并像素后计算，并非各图片 AUROC 的算术平均；总体指标还会比较不同织物、不同缺陷图片之间的像素分数。逐图归一化破坏了跨图片的绝对可比性，因此可能出现类别内部排序较好、跨类别总体排序接近随机的现象。异常像素占比极低时，AUROC 还会受到大量容易区分的背景像素影响；Pixel AP 更关注高分区域的精确性。本实验几乎所有类别的 AP 都很低，说明许多高响应仍落在正常纹理或图像边界，而非真实缺陷上。

8.4 域内训练案例分析

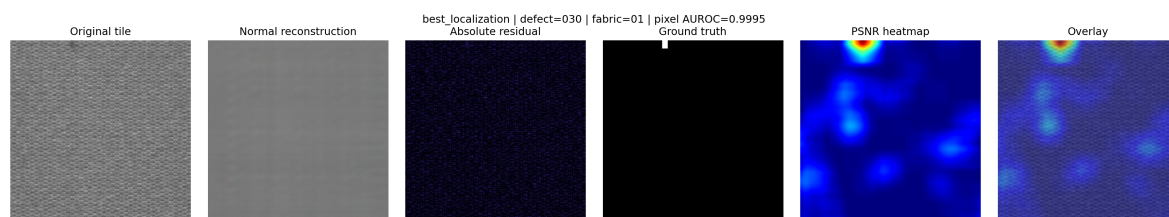


图 10 AITEX 域内训练后的较好案例：棉结缺陷 0093_030_01，所示图块对应原图单图 Pixel AUROC 为 0.9995、Pixel AP 为 0.6833。热图峰值与上边缘处的真实缺陷基本重合，但图内仍有多个较弱伪响应。

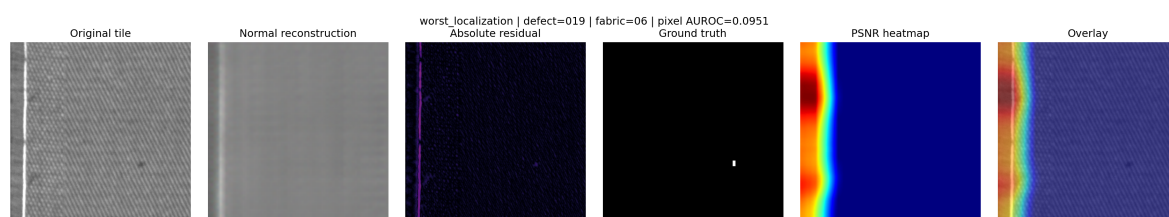


图 11 AITEX 域内训练后的失败案例：毛球缺陷 0058_019_06，单图 Pixel AUROC 为 0.0951、Pixel AP 约为 3.05×10^{-5} 。真实缺陷是中间偏右的小区域，热图却主要响应左侧织物边缘，说明边界重建误差压过了细小缺陷信号。

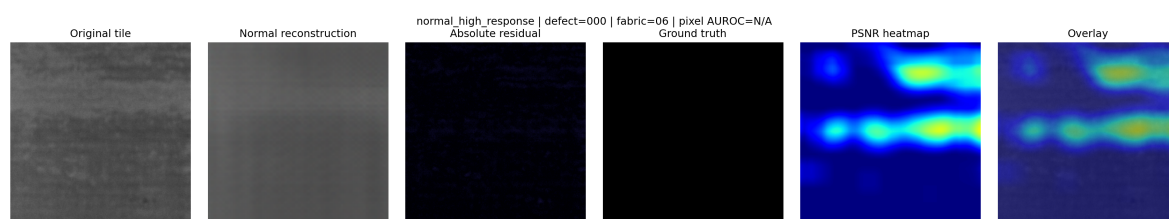


图 12 AITEX 域内训练后的正常高响应案例 0017_000_06。该图不存在真实缺陷，但周期纹理和亮度带仍产生连续高响应；逐图 min-max 归一化进一步放大了这种相对误差。

8.5 域内实验小结

AITEX 域内训练证明了两个层面的结论。第一，目标域训练能显著修复图片级异常分数的排序，最大值 AUROC 从 0.4463 提升到 0.8173，因此零样本失败主要包含明

显的域偏移因素。第二，图片级改进没有传递到像素级定位：整体 Pixel AUROC 只有 0.5213，案例中同时存在准确定位、边界误响应和正常纹理高响应。当前模型可以较好回答“整张图是否异常”，但还不能稳定回答“异常具体在哪里”。

9 结果讨论

9.1 与论文结果的差异

表 25 结果差异分析

差异来源	本项目情况	可能影响
框架与 CUDA 版本 数据预处理	PyTorch 2.11.0+cu128；论文为 PyTorch 1.8.0、CUDA 11.4 固定四块裁剪后缩放	算子实现、初始化与随机性可能改变训练轨迹 局部缺陷可能被放大，但跨块上下文会丢失
随机种子 论文未公开细节	当前仅种子 42 四块聚合、MF 具体取层、TS-GAN 异常图名单等由本项目明确设定	无法估计均值和方差 可能造成指标与论文不完全可比
测试集组成	TS-GAN 排除 10 张异常训练图	与其他模型不能直接严格比较

9.2 为什么 AITEX 域内结果仍低于 Grid

AITEX 已经参与重新训练，但图片级最大聚合 AUROC 为 0.8173，仍低于 Grid 的 0.9534；整体 Pixel AUROC 为 0.5213，也明显低于 Grid 的 0.9683。这个差距不能简单归结为“服务器训练失败”或“模型没有学到东西”：AITEX 域内结果相对零样本已有显著提升，说明训练确实有效。更合理的解释是数据规模、任务难度、预处理与评价协议共同造成了剩余差距。

表 26 AITEX 域内结果仍低于 Grid 的可能原因

原因	实验依据	可能影响
正常训练数据更少且更多样	Grid 有 264 张同一类别正常图；AITEX 只有 84 张正常训练图，却覆盖 7 种织物，每种仅 12 张	单个织物域可供学习的正常纹理不足，生成器更难同时重建多种材质、亮度和纹理周期
异常监督覆盖不足	Grid 使用 10 张异常图覆盖 5 类、每类 2 张；AITEX 对 9 个已见类型各只有 1 张异常训练图，另有 3 个类型训练阶段完全未见	一张样本难以表达同类缺陷的位置、大小和外观变化，异常支路容易学习到个例而非稳定类别规律
缺陷更细小且类别不均衡	AITEX 中 019 有 38 张测试图，而多个类型仅 1-2 张；不少真实掩码只是极小点或细线	大类主导总体结果，稀有类别指标不稳定；细小缺陷在缩放后更容易丢失
长图切块与低分辨率	AITEX 长图先切为 256×256 图块，再缩放到 64×64 ；案例中织物边缘产生强重建误差	缺陷可能被缩小到少量像素，图块边缘和亮度带反而成为更强的异常响应
参数直接沿用 Grid	学习率、损失权重、潜变量维度和 150+50 轮训练计划基本沿用 Grid，没有使用 AITEX 测试标签重新调参	保证了测试独立性，但这些超参数不一定适合多织物目标域，当前结果可能尚未达到 AITEX 上限
测试协议并不相同	Grid 测试为 21 张正常图和 47 张异常图；AITEX 域内测试为 29 张正常图和 97 张异常图，缺陷组成、难度和图像结构均不同	两个 AUROC 可用于描述各自协议下的表现，但不能把差值全部解释为模型训练质量差异
像素热图未使用异常支路	图片级 TS-GAN 分数联合正常与异常支路；PSNR 热图只使用正常生成器，并对每张图单独 min-max 归一化	第二阶段异常监督带来的图片级提升不会直接传递到定位；正常纹理和边缘响应还会被逐图归一化放大

因此，AITEX 图片级 AUROC 低于 Grid，主要说明 AITEX 是一个样本更少、纹理域更多、缺陷尺度更细且类别分布更不均衡的任务，同时当前超参数和输入分辨率仍沿用了 Grid 设定。像素级差距更大，则还叠加了“像素热图只用正常支路”和“逐图归一化破坏跨图标尺”两个因素。现有结果支持目标域训练有效，但不支持把 0.8173 与 0.9534 的差距解释成某一个单独原因；后续需要通过消融实验逐项验证。

9.3 模型优势与不足

- **GANomaly**: 结构和评分逻辑清晰，适合作为只用正常数据的基线；但仅以潜变量差异评分，可能忽略局部重建异常，对细小纹理缺陷较弱。
- **MF-GANomaly**: 在基线中加入像素和浅层特征，使局部纹理变化更容易进入损失与评分；代价是损失量级、取层和权重更敏感，并增加特征计算。
- **Mask Encoders**: 遮挡补全减少了直接复制输入的可能，本轮重建质量和 AUROC 均较好；但需要两阶段训练，中心遮挡任务与真实缺陷位置不完全一致，仍可能在边界或细小缺陷上失败。

- **TS-GAN**: 少量异常样本使模型显式学习异常方向, 本轮 Grid AUROC 最高; 但结果依赖异常样本是否具有代表性, 共享判别器在两阶段可能遗忘或饱和, 且其测试集与无监督模型不同。AITEK 零样本结果说明这种优势不会自动转化为跨数据域泛化能力; AITEK 域内训练又表明目标域样本可以明显改善图片级判别, 但异常支路的改进不会自动改善只依赖正常支路的 PSNR 像素热图。

9.4 实验有效性威胁

1. **内部有效性**: 当前同时报告最大与平均聚合, 没有据测试结果删去较差方案; TS-GAN 的 10 张异常训练图已从测试中排除。但此前开发过程查看过测试 AUROC, 因此后续独立数据才适合作为最终泛化证据。
2. **外部有效性**: AITEK-AFID 零样本验证扩展到了独立织物域, 域内实验则评估目标域训练后的能力; 但两者目前都只验证 TS-GAN, 且未覆盖其他物体类别、相机和光照条件, 因此仍不能将结论推广到一般工业异常检测。
3. **统计有效性**: 每个模型仅运行种子 42, 缺少均值、标准差、置信区间及显著性检验, 偶然初始化可能影响排序。
4. **实现有效性**: 论文未公开完整源码, 本项目对多尺度取层、判别器分类头、异常训练图和图片聚合做了可复现假设, 这些差异会影响与论文数值的严格比较。

10 结论与后续工作

10.1 主要结论

1. 四个模型已完成从数据读取、训练、检查点保存到图片级评估的完整流程; 在 Grid、固定四块裁剪和种子 42 下, 三个改进模型的 AUROC 均高于 GANomaly 基线。
2. MF-GANomaly 的最佳 AUROC 为 0.7703, Mask Encoders 为 0.9365; 与 GANomaly 的 0.7026 相比, 结果支持浅层特征约束与遮挡补全在本实验协议下有效。
3. TS-GAN 的最佳图片级 AUROC 为 0.9534, 但它使用 10 张异常训练图且测试样本数为 68, 不能与其余模型作完全公平的直接排名。按论文描述实现滑窗 PSNR 热图后, 额外 Pixel AUROC 为 0.9683, 说明该方法在当前 Grid 测试集上具有较好的像素排序能力; 该数值不是论文报告值, 且窗口参数仍属于复现假设。
4. 在完全未重训练的 AITEK-AFID 外部验证中, TS-GAN 图片级最大、平均汇总 AUROC 分别为 0.4463 和 0.5482, Pixel AUROC 为 0.7472、Pixel AP 为 0.0060。

与 Grid 结果的明显落差证明当前模型存在较强域依赖：局部定位排序能力部分保留，但整图异常识别和精确像素定位不能直接迁移。

5. 在 AITEX 固定划分上重新训练后，图片级最大聚合 AUROC 提升至 0.8173、AP 提升至 0.9368，说明目标域训练能够显著缓解整图判别的域偏移；但整体 Pixel AUROC 降至 0.5213、Pixel AP 为 0.0049。图片级检测与像素级定位由不同信号驱动，前者的提升不能被解释为后者也已解决。

10.2 后续工作：可以进一步尝试的事项

- **热图归一化与校准消融：**在不重新训练模型的条件下，比较逐图 min-max、由正常验证集确定的全局缩放范围、正常像素分位数校准和边界响应抑制。重点观察 Pixel AUROC、Pixel AP 与正常图高响应是否改善，以验证当前定位失败究竟来自模型还是后处理标尺。
- **滑动窗口敏感性：**比较窗口边长 4、8、16，步长 1、2、4，以及不同重叠窗口响应汇总方式，同时报告定位指标与运行时间。由于论文未公开这些参数，该实验可量化复现假设对结论的影响。
- **多随机种子：**至少运行种子 7、42、2026，优先重复 TS-GAN Grid 与 AITEX 域内实验，报告均值和标准差；若算力允许，再扩展到其余三种模型，以判断当前结果是否依赖偶然初始化和数据划分。
- **AITEX 无监督对照：**将现有使用 9 张异常训练图的结果作为弱监督基线，再增加只使用 84 张正常训练图的目标域适配，区分正常纹理域适配与少量异常监督各自带来的贡献。
- **定位结构改进：**针对正常高响应和细小缺陷漏定位，尝试更高输入分辨率、多尺度窗口和边界抑制；所有方案均应在正常验证集上确定参数，不能根据异常测试标签反复选择最佳方案。
- **统一公平比较：**在固定测试集、预处理和评价脚本的前提下比较模型，并明确区分 Grid 主实验、AITEX 零样本实验和 AITEX 域内实验，避免把不同数据协议下的指标直接排名。
- **工程归档：**将最终代码合并到主分支，保留配置、固定划分清单、指标文件和代表图；大型检查点单独归档而不提交 Git，并清理临时压缩包和重复运行目录。

参考文献

- [1] 步兆军. 基于自监督与弱监督学习的图像异常检测研究 [D]. 南京: 东南大学, 2022.
- [2] Akcay S, Atapour-Abarghouei A, Breckon T P. GANomaly: Semi-supervised anomaly detection via adversarial training[C]//Asian Conference on Computer Vision. Cham: Springer, 2018: 622–637.
- [3] Bergmann P, Fauser M, Sattlegger D, et al. MVTec AD: A comprehensive real-world dataset for unsupervised anomaly detection[C]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition. 2019: 9592–9600.
- [4] Silvestre-Blanes J, Albero-Albero T, Miralles I, et al. AFID: A public fabric image database for defect detection[J]. AUTEX Research Journal, 2019. 数据集官网: <https://www.aitex.es/afid/>.

A 关键代码整理与理解

本附录按“公共实现只解释一次、各模型只列差异”的原则整理代码。GANomaly是其余模型的参照；MF-GANomaly 复用其解码器，Mask Encoders 与 TS-GAN 复用相同的卷积块和对抗训练思想，因此不重复粘贴样板层定义。代码均摘自本项目实际运行版本，为控制篇幅省略类型标注、日志和异常处理，但不改变数据流与优化逻辑。

A.1 代码片段记录表

表 27 关键代码索引

文件与函数	输入输出	核心操作	自己的理解
<code>models.py:</code> <code>Generator</code>	图像、重建 图与两组隐 变量	编码-解码-再编 码	四个模型比较中的公 共基线
<code>models_mf.py:</code> <code>MFGenerator</code>	基线输出加 两组浅层特 征	暴露第一层特征 并参与损失	解码器参考 GANomaly, 不重复 列出
<code>models_mask.py:</code> <code>MaskGenerator</code>	128×128 图 像到重建图	六层编码器与对 称解码器	去掉第二编码器, 学 习遮挡补全
<code>models_ts.py:</code> <code>TSGenerator</code>	64×64 图像 到重建图	单次跳跃连接	正常流和异常流结构 相同、参数独立
<code>training.py:</code> <code>train_one_batch</code>	一个批次到 损失字典	先更新判别器, 再更新生成器	各模型训练器沿用该 交替框架
<code>train_mask.py/</code> <code>train_ts.py</code>	数据集、阶 段与检查点	阶段切换和参数 冻结	体现两阶段训练协 议, 而非新网络层
<code>losses*.py/</code> <code>evaluate*.py</code>	重建与特征 到异常分数	分支评分及块级 聚合	训练损失与最终 AUROC 的接口

A.2 GANomaly: 公共基线网络

GANomaly 的生成器由两个结构相同但参数独立的编码器和一个解码器组成。输入 $x \in \mathbb{R}^{B \times 3 \times 64 \times 64}$ 经第一个编码器得到 $z \in \mathbb{R}^{B \times 128 \times 1 \times 1}$, 解码为 $\hat{x}$, 再由第二个编码器得到 $\hat{z}$ 。卷积层的具体堆叠属于常规 DCGAN 写法, 此处保留决定算法含义的前向传播。

Listing 1 GANomaly 生成器的核心前向传播

```

1 class Generator(nn.Module):
2     def __init__(self, image_channels=3, latent_dim=128):
3         super().__init__()
4         self.encoder1 = Encoder(image_channels, latent_dim)
5         self.decoder = Decoder(image_channels, latent_dim)
6         self.encoder2 = Encoder(image_channels, latent_dim)
7
8     def forward(self, images):
9         latent_original = self.encoder1(images)
10        reconstructed = self.decoder(latent_original)
11        latent_reconstructed = self.encoder2(reconstructed)
12        return reconstructed, latent_original, latent_reconstructed

```

这三项输出分别服务于像素重建损失、潜变量一致性损失和判别器特征匹配损失。两个编码器不能写成同一个对象，否则它们会共享参数，便不再是论文所述的“原图编码”和“重建图编码”两条映射。

A.3 MF-GANomaly: 只列相对基线的增量

MF-GANomaly沿用代码 1 的编码-解码-再编码骨架，并直接复用 `models.py` 中的 `Decoder`。它的关键增量是同时返回第一层浅层特征，使细节纹理差异能够参与重建项；判别器的真假判断与特征输出同 GANomaly，因此不再重复列出。

Listing 2 MF-GANomaly 暴露浅层特征并组成五项输出

```

1 class MFEncoder(Encoder):
2     def forward(self, images):
3         shallow = self.conv1(images)
4         features = self.conv2(shallow)
5         features = self.conv3(features)
6         features = self.conv4(features)
7         latent = self.conv5(features)
8         return latent, shallow
9
10 class MFGenerator(nn.Module):
11     def forward(self, images):
12         z, shallow_original = self.encoder1(images)
13         reconstructed = self.decoder(z)
14         z_hat, shallow_reconstructed = self.encoder2(reconstructed)

```

```

15     return MFOutput(reconstructed, z, z_hat,
16                     shallow_original, shallow_reconstructed)

```

浅层张量保持计算图，代码中没有 `detach()`，所以浅层特征距离能够反向更新两个编码器。其异常分数为

$$S_{\text{MF}} = \|z - \hat{z}\|_2 + \alpha (\|x - \hat{x}\|_2 + \|f_1(x) - f_1(\hat{x})\|_2), \quad (32)$$

其中本实验取 $\alpha = 40$ 。这说明 MF-GANomaly 不是简单增加一层网络，而是把较低层的纹理误差显式加入训练和检测。

A.4 Mask Encoders: 遮挡补全与两阶段迁移

Mask Encoders 使用 128×128 输入、512 维瓶颈和对称解码器。其 down/up 卷积块与前述 DCGAN 结构重复，关键差异在训练数据：第一阶段把中心 64×64 区域置为归一化空间中的黑色 -1，以遮挡图为输入、完整图为目标；第二阶段载入第一阶段的生成器和判别器权重，再用完整正常图执行重建微调。

Listing 3 Mask Encoders 的中心遮挡样本

```

1 def center_mask(image, mask_size=64):
2     masked = image.clone()
3     height, width = image.shape[-2:]
4     top = (height - mask_size) // 2
5     left = (width - mask_size) // 2
6     masked[..., top:top + mask_size, left:left + mask_size] = -1.0
7     return masked
8
9 def __getitem__(self, index):
10     sample = self.base[self.indices[index]]
11     complete = sample["image"]
12     return {**sample, "image": center_mask(complete, self.mask_size),
13            "target": complete}

```

Listing 4 Mask Encoders 的两阶段权重衔接

```

1 components = build(config, device)
2 pretrain_data = MaskPretrainDataset(...)
3 train_phase("stage1", pretrain_epochs, pretrain_data,
4             components, config, device, pretrain_dir)
5

```

```

6 pretrained = torch.load(pretrain_dir / "checkpoint_final.pt")
7 components = build(config, device) # reset Adam states
8 components[0].load_state_dict(pretrained["generator"])
9 components[1].load_state_dict(pretrained["discriminator"])
10 finetune_data = MVTecFourCropTrainDataset(...)
11 train_phase("stage2", finetune_epochs, finetune_data,
12             components, config, device, finetune_dir)

```

第二阶段重新创建 Adam，说明迁移的是网络参数而非第一阶段的动量状态。生成器只有“编码器-解码器”，没有 GANomaly 的第二编码器；检测时直接以 $\|x - \hat{x}\|_2$ 作为块级异常分数。

A.5 TS-GAN：单跳连、双生成器与共享判别器

TS-GAN 的正常流 G_N 与异常流 G_A 都采用同一 `TSGenerator` 类，但实例独立。生成器从编码器最后一个 4×4 特征图向解码端加入一次通道拼接，以保留空间纹理；二者共享一个判别器 D 。

Listing 5 TS-GAN 单次跳跃连接的前向传播

```

1 def forward(self, images):
2     e1 = self.e1(images)
3     e2 = self.e2(e1)
4     e3 = self.e3(e2)
5     e4 = self.e4(e3) # [B, 512, 4, 4]
6     z = self.e5(e4) # [B, 512, 1, 1]
7     decoded = self.d1(z) # [B, 512, 4, 4]
8     decoded = torch.cat([decoded, e4], dim=1)
9     reconstructed = self.d5(self.d4(self.d3(self.d2(decoded))))
10    return reconstructed

```

Listing 6 TS-GAN 两条流的分阶段训练

```

1 normal_g = TSGenerator(...)
2 abnormal_g = TSGenerator(...)
3 discriminator = TSDiscriminator(...)
4 models = normal_g, abnormal_g, discriminator
5
6 set_requires_grad(abnormal_g, False)
7 run_phase("normal", normal_g, normal_epochs, normal_data,
8           normal_lr, models, config, device, normal_dir)

```

```

9
10 set_requires_grad(normal_g, False)
11 set_requires_grad(abnormal_g, True)
12 run_phase("abnormal", abnormal_g, abnormal_epochs, anomaly_data,
13          abnormal_lr, models, config, device, abnormal_dir)

```

第一阶段只更新 G_N 与共享 D ，第二阶段冻结 G_N 、更新 G_A 与同一个 D 。因此“双流”不是把两张图同时送入一个网络，而是得到两个不同训练分布下的重建器，测试时再组合两条分支的分数。

A.6 公共训练核心：先更新判别器，再更新生成器

四个模型的训练器虽然输出字段不同，但都遵循代码 7 的交替更新，因此这里只保留 GANomaly 的完整关键路径。其余模型只改动接口：MF-GANomaly 使用代码 2 的五项输出，Mask Encoders 将输入与目标分开，TS-GAN 将潜变量项换成判别器特征项和对抗项。

Listing 7 单个批次中的判别器-生成器交替更新

```

1 # 1. Update D; G is used only to produce detached fake images.
2 set_requires_grad(discriminator, True)
3 optimizer_d.zero_grad(set_to_none=True)
4 with torch.no_grad():
5     reconstructed_for_d, _, _ = generator(images)
6 real_prob, _ = discriminator(images)
7 fake_prob, _ = discriminator(reconstructed_for_d.detach())
8 d_losses = discriminator_loss(real_prob, fake_prob)
9 d_losses["total"].backward()
10 optimizer_d.step()
11
12 # 2. Freeze D parameters, but keep the path from D back to G.
13 set_requires_grad(discriminator, False)
14 optimizer_g.zero_grad(set_to_none=True)
15 reconstructed, z, z_hat = generator(images)
16 with torch.no_grad():
17     _, real_features = discriminator(images)
18     _, fake_features = discriminator(reconstructed)
19 g_losses = generator_loss(images, reconstructed, z, z_hat,
20                          real_features, fake_features)
21 g_losses["total"].backward()

```

```

22 optimizer_g.step()
23 set_requires_grad(discriminator, True)

```

判别器阶段对重建图使用 `no_grad()` 和 `detach()`，防止判别器损失更新生成器。生成器阶段则只冻结判别器参数，并未对重建分支断开计算图，所以特征匹配损失仍能穿过判别器回传至生成器。这个差别是实现是否正确的关键。

A.7 损失函数、异常评分与整图聚合

表 28 汇总四种模型真正用于优化和排序的量。所有距离均按单个样本展平后计算，再对批次求均值，避免把不同样本混在同一个范数中。

表 28 四种模型的训练目标与测试异常分数

模型	生成器训练项	测试时块级异常分数
GANomaly	像素 L1、隐变量 MSE、判别器特征 MSE，权重 40:1:1	$\ z - \hat{z}\ _2$
MF-GANomaly	像素 L2、浅层特征 L2、隐变量 L2、判别器特征 L2	$\ z - \hat{z}\ _2 + 40(\ x - \hat{x}\ _2 + \ f_1 - \hat{f}_1\ _2)$
Mask Encoders	完整目标与重建的 L2，加真假 BCE	$\ x - \hat{x}\ _2$
TS-GAN	每条流均使用像素 L2、共享判别器特征 L2 与真假 BCE	$S_N - \eta S_A$ ，其中 $S_N = \alpha L_{px,N} + L_{ft,N}$ ， $S_A = \beta L_{px,A} + L_{ft,A}$

Listing 8 TS-GAN 分支评分和四块整图聚合

```

1 def branch_score(images, reconstructed, real_features,
2     fake_features, pixel_weight):
3     pixel = l2_per_sample(images, reconstructed)
4     feature = l2_per_sample(real_features, fake_features)
5     return pixel_weight * pixel + feature
6
7 sn = branch_score(images, normal_recon, real_f, normal_f, alpha)
8 sa = branch_score(images, abnormal_recon, real_f, abnormal_f, beta)
9 patch_scores = sn - eta * sa
10
11 image_score_max = max(four_patch_scores)
12 image_score_mean = sum(four_patch_scores) / 4.0

```

本实验设 $\alpha = 40$ 、 $\beta = 10$ 、 $\eta = 0.1$ 。每张原图被四块裁剪后，分别用最大值和均值形成整图分数，再将该连续分数与正常/异常标签输入 `roc_auc_score`。因此 AUROC

评价的是排序能力；阈值只在计算准确率、精确率、召回率和 F1 时使用，并不参与 AUROC 本身。

B 完整配置与复现实验命令

B.1 配置文件

Listing 9 配置摘录

```
1 experiment:
2   seed: 42
3 dataset:
4   category: grid
5   preprocessing: four_crops
6   batch_size: 16
7 common_optimizer:
8   name: Adam
9   beta1: 0.5
10  beta2: 0.999
11 ganomaly:
12   image_size: 64
13   latent_dim: 128
14   epochs: 150
15   learning_rate: 0.0002
16 mf_ganomaly:
17   image_size: 64
18   latent_dim: 128
19   epochs: 150
20   reconstruction_metric: l2
21 mask_encoders:
22   image_size: 128
23   mask_size: 64
24   pretrain_fraction: 0.5
25   epochs: [80, 80]
26 ts_gan:
27   image_size: 64
28   latent_dim: 512
29   anomaly_images_per_type: 2
30   epochs: [150, 50]
```

```

31 learning_rates: [0.0002, 0.0001]
32 evaluation: {alpha: 40, beta: 10, eta: 0.1}

```

B.2 运行命令

Listing 10 训练和评估命令

```

1 python -m anomaly_reproduction.train_four_crops
2 python -m anomaly_reproduction.evaluate --config configs/
   ganomaly_grid_four_crops.yaml
3 python -m anomaly_reproduction.train_mf
4 python -m anomaly_reproduction.evaluate_mf
5 python -m anomaly_reproduction.train_mask
6 python -m anomaly_reproduction.evaluate_mask
7 python -m anomaly_reproduction.train_ts
8 python -m anomaly_reproduction.evaluate_ts

```

B.3 结果文件清单

表 29 结果文件与用途

文件	用途
runs/<experiment>/checkpoint_final.pt	模型参数、训练轮次与最终配置
runs/<experiment>/history.csv	每轮生成器、判别器及分项损失
runs/<experiment>/test_metrics.yaml	最终图片级 AUROC 等汇总指标
runs/<experiment>/test_patch_scores.csv	每个裁剪块的标签与异常分数
runs/<experiment>/test_image_scores.csv	四块重新聚合后的整图分数
runs/<experiment>/test_reconstruction_preview.png	原图、重建图与残差预览；正式独立案例仍待本人筛选